

# **C# / .NET Learning Roadmap**

Topic Notes — Section 1: C# Language Fundamentals

Topics: 1.1 through 1.10

Date: 29 June 2026

## Section 1 — C# Language Fundamentals

This section covers the core C# language features you will use in every project. Topics are ordered by dependency — each builds on what came before.

### 1.1. .NET Ecosystem Overview (CLR, BCL, SDK)

---

**Builds on:** Nothing in C# yet — this is orientation before you write a single line. Think of it as a map of the territory.

#### What is the .NET Ecosystem and why does it exist?

When you write C# code, you're not writing instructions the CPU understands directly. You're writing in a high-level language that needs a whole supporting infrastructure to run. That infrastructure is .NET.

Three layers make it work:

#### Layer 1 — CLR (Common Language Runtime)

The CLR is the engine that runs your code. When you build a C# project, it doesn't compile to machine code directly — it compiles to an intermediate format called IL (Intermediate Language). The CLR takes that IL and executes it.

What the CLR handles so you don't have to:

- Memory management — allocating and freeing memory (garbage collection)
- Type safety — catching type mismatches at runtime
- Exception handling — the try/catch infrastructure
- Thread management — running code concurrently

Think of the CLR as the virtual machine that your C# program lives inside.

```
Your C# Code
    | (compiler)
IL Code (.dll)
    | (CLR at runtime)
Machine Code -> CPU executes
```

#### Layer 2 — BCL (Base Class Library)

The BCL is the standard library that ships with .NET — thousands of pre-built classes covering everything from string manipulation to file I/O to HTTP requests to collections.

You use BCL types constantly without thinking about it:

```
// All of these are BCL types:
string name = "Sathish";           // System.String
List<int> nums = new List<int>();   // System.Collections.Generic.List<T>
Console.WriteLine("Hello");        // System.Console
DateTime now = DateTime.Now;       // System.DateTime
```

In ASP.NET Core specifically, BCL types like `HttpClient`, `ILogger`, `CancellationToken` appear everywhere.

### Layer 3 — SDK (Software Development Kit)

The SDK is the toolchain you install on your machine to build .NET applications. It includes:

- The compiler (csc) — turns your C# into IL
- The CLI (dotnet) — the command-line tool you use for everything
- Project templates — scaffolding for new projects
- Build tools (MSBuild) — compiles, links, and packages your app

```
dotnet new webapi -n MyApi      # create a new Web API project
dotnet build                    # compile the project
dotnet run                     # build and run
dotnet test                     # run tests
dotnet publish                  # package for deployment
```

The SDK is what you install. The CLR is what runs your app (it's included in the runtime, a subset of the SDK).

### How they relate

```
SDK -> you use this to BUILD
CLR -> this RUNS your built app
BCL -> these are the LIBRARIES available to your code
```

In production, a server only needs the runtime (CLR + BCL) installed — not the full SDK. Your CI/CD pipeline uses the SDK to build, the server uses the runtime to run.

### .NET vs .NET Framework — one thing to know

You'll see both terms. They're different:

|                      | .NET (5,6,7,8,9...)   | .NET Framework (4.x) |
|----------------------|-----------------------|----------------------|
| Cross-platform       | Yes (Win, Mac, Linux) | Windows only         |
| Status               | Active, current       | Legacy, maintenance  |
| Use for new projects | Yes                   | No                   |

When a job posting says ".NET developer," they mean modern .NET. .NET Framework is legacy — you may encounter it in older codebases but you don't build new things with it.

### Where this shows up in real work

- Every .csproj file specifies which .NET version your project targets (net9.0)
- When a NuGet package says "supports .NET 6+," it means CLR/BCL compatibility
- When you install .NET on a deployment server, you're installing the runtime (CLR + BCL)
- dotnet CLI commands are what your CI/CD pipeline runs

### Common Misconceptions

- **C# and .NET are the same thing.** C# is the language. .NET is the platform/runtime. C# compiles to IL that runs on the CLR. You could theoretically write .NET apps in F# or VB.NET — same CLR, different language.

- **I need to understand IL to be productive.** You don't. IL is an implementation detail. Understanding that it exists explains why .NET is cross-platform and why the CLR can do garbage collection.
- **.NET Framework and .NET are interchangeable.** They're not — different runtimes, different APIs, different platform support. Always clarify which one a codebase or tutorial targets.
- **The SDK is what runs my app in production.** No — you install the lighter runtime on production servers, not the full SDK.

**Self-check — you should now be able to explain:**

- What the CLR does and why your C# doesn't compile directly to machine code
- What the BCL is and name three BCL types you'll use constantly
- What the SDK is and what dotnet build / dotnet run actually do
- The difference between .NET (modern) and .NET Framework (legacy)

## 1.2. Environment Setup, Project/Solution Structure

---

**Builds on:** 1.1 — you now know the SDK gives you the dotnet CLI and the compiler. This topic is about what gets created when you actually use them.

**What needs to be installed?**

Three things:

**1. .NET SDK** — download from [dotnet.microsoft.com](https://dotnet.microsoft.com). One install gives you the CLI, compiler, and runtime.

```
# Verify installation
dotnet --version      # e.g. 9.0.100
dotnet --list-sdks    # all installed SDKs
```

**2. An IDE** — Visual Studio (Windows, full-featured, free Community edition) or Rider (cross-platform, paid). VS Code with the C# Dev Kit extension works too but is lighter.

**3. A database** (later) — SQL Server, PostgreSQL, or SQLite when you reach EF Core. Not needed yet.

**Project vs Solution — the key distinction**

These two terms appear constantly and confuse beginners.

**Project** = one compilable unit. One .csproj file. Produces one output — a .dll or an executable.

**Solution** = a container for one or more projects. One .sln file. No code of its own — just organises projects and their references.

```
MyApp.sln                                <- solution file
|
+-- MyApp.API/
|   +-- MyApp.API.csproj                 <- project file (Web API)
|   +-- Program.cs
```

```

|   +-- Controllers/
|
+-- MyApp.Application/
|   +-- MyApp.Application.csproj <- project file (business logic)
|   +-- Services/
|
+-- MyApp.Domain/
|   +-- MyApp.Domain.csproj      <- project file (entities)
|   +-- Entities/
|
+-- MyApp.Tests/
|   +-- MyApp.Tests.csproj      <- project file (unit tests)
|   +-- UserServiceTests.cs

```

This multi-project structure maps directly to the Clean Architecture you'll implement in section 17.

## Creating a project from scratch

```

# Create a solution
dotnet new sln -n MyApp

# Create a Web API project
dotnet new webapi -n MyApp.API

# Add project to solution
dotnet sln MyApp.sln add MyApp.API/MyApp.API.csproj

# Reference one project from another
dotnet add MyApp.API/MyApp.API.csproj reference MyApp.Application/MyApp.Application.csproj

```

## Anatomy of a .csproj file

The .csproj is an XML file that defines everything about your project. You'll edit it directly sometimes.

```

<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <TargetFramework>net9.0</TargetFramework>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
  </PropertyGroup>

  <ItemGroup>
    <!-- NuGet packages your project depends on -->
    <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.0" />
  </ItemGroup>

  <ItemGroup>
    <!-- References to other projects in your solution -->
    <ProjectReference Include="..\MyApp.Application\MyApp.Application.csproj" />
  </ItemGroup>

</Project>

```

Key properties:

- TargetFramework — which .NET version to compile for
- Nullable — enables nullable reference type warnings (section 1.14)
- ImplicitUsings — auto-includes common namespaces so you don't write using System; everywhere

## What a new Web API project contains

```

MyApp.API/
+-- MyApp.API.csproj      <- project config
+-- Program.cs            <- app entry point (section 1.3)
+-- appsettings.json      <- configuration (section 12.4)
+-- appsettings.Development.json
+-- Controllers/
|   +-- WeatherForecastController.cs <- sample controller
+-- Properties/
    +-- launchSettings.json <- local dev URLs and ports

```

launchSettings.json is local-only — never committed to source control as a source of truth. It controls what URL your app starts on when you hit dotnet run.

## The build output folder

When you run dotnet build, compiled output goes into:

```

bin/
+-- Debug/
    +-- net9.0/
        +-- MyApp.API.dll      <- your compiled code as IL
        +-- MyApp.API.exe      <- executable wrapper (Windows)
        +-- MyApp.API.deps.json <- dependency manifest

```

obj/ is a temp folder used during build — never touch it manually. Both bin/ and obj/ go in .gitignore.

## .gitignore essentials for .NET

```

bin/
obj/
*.user
.vs/
appsettings.Development.json # sometimes — if it has local secrets

```

The dotnet new command generates a .gitignore automatically — use it.

## Common Misconceptions

- **One project = one solution.** For small apps, yes. But real-world .NET solutions almost always have multiple projects — API, Application, Domain, Tests are standard. Don't put everything in one project.
- **The .sln file is where I configure my app.** No — .sln is just a list of projects. Configuration lives in .csproj and appsettings.json.
- **I should edit files in bin/ or obj/.** Never. Those are generated outputs — they get wiped and regenerated on every build.

- **launchSettings.json controls production deployment.** It's a local dev convenience only. Production hosting is configured separately (environment variables, Docker, IIS, etc.).

#### Self-check — you should now be able to explain:

- The difference between a project and a solution, with an example of when you'd have multiple projects
- What the .csproj file controls and what three things you'd commonly find in it
- Which folders go in .gitignore and why
- What dotnet new webapi, dotnet build, and dotnet run each do

## 1.3. Main Method, Top-Level Statements

---

**Builds on:** 1.2 — you know a project has a Program.cs as its entry point. This topic explains exactly what that file is and how .NET knows where to start running your code.

#### The problem this solves

Every program needs a starting point — a specific place where execution begins. In .NET, that's historically been the Main method. Without a defined entry point, the runtime wouldn't know which line of code to run first.

#### The Classic Main Method (pre .NET 6)

Before .NET 6, every console or web app required this boilerplate:

```
using System;

namespace MyApp
{
    class Program
    {
        static void Main(string[] args)
        {
            Console.WriteLine("Hello, World!");
        }
    }
}
```

Rules that had to be followed exactly:

- Must be inside a class
- Must be named Main (capital M)
- Must be static
- Return type: void or int
- Optional parameter: string[] args (command-line arguments)

This was the only way to define an entry point. A lot of ceremony for "start here."

#### Top-Level Statements (.NET 6+)

.NET 6 introduced top-level statements — you can write executable code directly in a file, no class or Main method required. The compiler generates the boilerplate for you behind the scenes.

```
// Program.cs – this IS the entry point, no class needed
Console.WriteLine("Hello, World!");
```

That's a complete, runnable .NET program. The compiler wraps it in a Main method internally — you just don't see it.

With arguments and async:

```
// args is still available as a magic variable
Console.WriteLine($"Args count: {args.Length}");

// async is supported too
await Task.Delay(1000);
Console.WriteLine("Done");
```

### What Program.cs looks like in a real ASP.NET Core app

In a Web API project, Program.cs is the entry point and also where you configure the entire app:

```
var builder = WebApplication.CreateBuilder(args);

// Register services into the DI container
builder.Services.AddControllers();
builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

var app = builder.Build();

// Configure the middleware pipeline
if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();
app.UseAuthorization();
app.MapControllers();

app.Run();
```

This is top-level statements in practice — no class, no explicit Main, just code that runs top to bottom. You'll work with this file constantly from section 12 onwards.

### The two sections of Program.cs

Notice the pattern above has a clear split:

```
Everything BEFORE app.Build() -> registering services (what your app needs)
Everything AFTER app.Build() -> configuring the pipeline (how requests are handled)
```

This distinction matters a lot when you reach Dependency Injection (section 12.2) and Middleware (section 13.6). For now, just recognise the two-phase structure.



## Can you still use the classic Main method?

Yes — both styles are valid. The classic style is still common in older codebases and some developers prefer its explicitness. Top-level statements are the default in all dotnet new templates from .NET 6 onwards.

One rule: only one file in a project can have top-level statements. The compiler generates exactly one Main — you can't have two entry points.

## Minimal APIs (a preview)

Top-level statements also enable Minimal APIs — defining HTTP endpoints directly in Program.cs without controllers:

```
var builder = WebApplication.CreateBuilder(args);
var app = builder.Build();

app.MapGet("/api/users", () => new[] { "Sathish", "Priya" });

app.Run();
```

This is a complete Web API. You'll cover this properly in section 13.3 — just know it exists and that Program.cs is where it lives.

## Common Misconceptions

- **Top-level statements are less powerful than Main.** They're identical at runtime — the compiler generates the same IL either way. No capability is lost.
- **Program.cs is just boilerplate to ignore.** In ASP.NET Core it's the most important file in your project. Every service, middleware, and pipeline configuration lives here or flows from here.
- **I can have top-level statements in multiple files.** No — exactly one file per project. Attempting two causes a compile error.
- **args isn't available in top-level statements.** It is — it's implicitly declared by the compiler. You can use args directly without declaring it.
- **The two sections of Program.cs (before/after Build) are interchangeable.** They're not. Registering a service after Build() has no effect. Configuring middleware before Build() causes an error. Order matters.

## Practice Problem

Look at this Program.cs and identify what's wrong:

```
var app = WebApplication.CreateBuilder(args).Build();

app.Services.AddControllers(); // registering after Build

app.UseHttpsRedirection();
app.MapControllers();
app.Run();
```

What is the bug, why does it happen, and how would you fix it? Try it before reading on — the answer is in the misconceptions section above.

### Self-check — you should now be able to explain:

- What the classic Main method signature looks like and what each part means
- What top-level statements are and what the compiler does with them
- The two-phase structure of Program.cs in an ASP.NET Core app and why order matters
- Why you can only have top-level statements in one file per project

## 1.4. Variables, Data Types, Value vs Reference Types

**Builds on:** 1.3 — you have a running program. Now you need to store and work with data inside it.

### Variables — the basics

A variable is a named storage location for a value. In C#, every variable has a fixed type declared at compile time.

```
int age = 30;
string name = "Sathish";
bool isActive = true;
double salary = 85000.50;
```

Format: type variableName = value;

### The Built-in Value Types You'll Use Daily

| Type    | Size    | Range / Use                  | Example                      |
|---------|---------|------------------------------|------------------------------|
| int     | 32-bit  | -2.1B to 2.1B, whole numbers | int count = 10;              |
| long    | 64-bit  | Very large whole numbers     | long fileSize = 9999999999L; |
| double  | 64-bit  | Decimals, scientific         | double pi = 3.14159;         |
| decimal | 128-bit | Financial precision          | decimal price = 99.99m;      |
| bool    | 1-bit   | true / false                 | bool isValid = true;         |
| char    | 16-bit  | Single character             | char grade = 'A';            |
| float   | 32-bit  | Less precise decimal         | float temp = 36.6f;          |

Practical rule: Use int for whole numbers, decimal for money, double for everything else numeric, bool for flags, string for text.

### string — special but acts like a value type

string is technically a reference type (explained below) but behaves like a value type in everyday use because it's immutable — once created, it cannot be changed. Any "modification" creates a new string.

```
string name = "Sathish";
name = name + " Kumar"; // creates a NEW string, doesn't modify original
```

### The Critical Distinction — Value vs Reference Types

This is the most conceptually important part of this topic.

**Value types** store the actual data directly in the variable.

**Reference types** store a reference (memory address) pointing to where the data lives on the heap.

Think of it this way:

- Value type = the actual house
- Reference type = a piece of paper with the house's address written on it

### Value Types — copied on assignment

```
int a = 10;
int b = a;    // b gets a COPY of the value
b = 99;
```

```
Console.WriteLine(a); // 10 -- unchanged
Console.WriteLine(b); // 99
```

a and b are completely independent. Changing b has zero effect on a.

All the primitive types in the table above (int, double, bool, char, decimal) are value types. So are struct types.

### Reference Types — share the same object

```
// Person is a class (reference type)
var person1 = new Person { Name = "Sathish" };
var person2 = person1; // person2 gets a COPY of the REFERENCE
                      // both now point to the SAME object

person2.Name = "Priya";

Console.WriteLine(person1.Name); // "Priya" -- also changed!
Console.WriteLine(person2.Name); // "Priya"
```

person1 and person2 hold the same memory address — the same underlying object. Mutating through one variable mutates what the other sees.

Reference types: class, string, arrays, interface, delegate.

### Where they live in memory

| Stack (fast, automatic cleanup)                      | Heap (managed by GC) |
|------------------------------------------------------|----------------------|
| -----                                                | -----                |
| int a = 10            [ 10 ]                         |                      |
| int b = 20            [ 20 ]                         |                      |
|                                                      |                      |
| var p = new Person() [ ref ] -> { Name = "Sathish" } |                      |

Value types live on the stack — allocated and freed automatically as methods execute.

Reference types live on the heap — allocated when you use new, freed by the garbage collector (CLR's job from 1.1).

You don't manage this manually — just understand the consequence: value types are copied, reference types are shared.

## Realistic scenario — where this bites you in .NET

This comes up in API development when mapping DTOs:

```
public class OrderDto
{
    public List<string> Items { get; set; }
}

var dto1 = new OrderDto { Items = new List<string> { "Keyboard" } };
var dto2 = new OrderDto();
dto2.Items = dto1.Items; // REFERENCE copy -- same list!

dto2.Items.Add("Monitor");

Console.WriteLine(dto1.Items.Count); // 2 -- dto1 was affected!
```

This is a common source of bugs. If you want independent copies of reference type objects, you need to explicitly clone or copy them.

## Default values

Unassigned variables have defaults:

```
int      -> 0
double   -> 0.0
bool     -> false
char     -> '\0'
string   -> null    (reference type -- no object yet)
object   -> null
```

Local variables in methods must be assigned before use — the compiler enforces this.

Class-level fields get their defaults automatically.

## Common Misconceptions

- **string is a value type because it behaves like one.** It's a reference type. It just happens to be immutable and has special compiler support (`==` compares content, not reference). Don't assume value-type rules apply to it in all cases.
- **Changing a reference type variable in a method changes it everywhere.** Only if you mutate the object the reference points to. Reassigning the variable inside a method (pointing it to a new object) does NOT affect the caller's variable — the reference itself was copied.
- **Value types are always faster.** Usually true for small types, but large structs can be slower than references because copying 500 bytes is expensive. Don't optimise prematurely.
- **I need to manage heap memory manually.** No. The CLR's garbage collector handles it. `IDisposable` (section 8.2) exists for unmanaged resources (file handles, DB connections) — not general memory.
- **decimal and double are interchangeable.** They're not. `double` uses binary floating point and has rounding errors — never use it for money. `decimal` uses decimal floating point and is exact for financial calculations.

```
double d = 0.1 + 0.2;
Console.WriteLine(d);           // 0.30000000000000004 <- NOT 0.3
```

```
decimal m = 0.1m + 0.2m;  
Console.WriteLine(m);           // 0.3 <- exact
```

### Practice Problem

What does this print, and why?

```
int x = 5;  
int y = x;  
y += 10;  
  
var listA = new List<string> { "one", "two" };  
var listB = listA;  
listB.Add("three");  
  
Console.WriteLine(x);  
Console.WriteLine(y);  
Console.WriteLine(listA.Count);  
Console.WriteLine(listB.Count);
```

Write your expected output and explain each line before moving on.

### Self-check — you should now be able to explain:

- The difference between value types and reference types with a concrete example
- Why decimal and not double for financial values
- What happens in memory when you assign one reference type variable to another
- Why mutating a List inside a method can affect the caller's list

## 1.5. Type Conversion (Implicit, Explicit, Parse/TryParse)

---

**Builds on:** 1.4 — you have typed variables. Type conversion is what happens when you need a value of one type to work as another type.

### Why does type conversion exist?

C# is statically typed — every variable has a fixed type known at compile time. But real programs constantly need to move data between types: a user inputs text (string), you need a number (int). An API returns a double, you store it as decimal. Without conversion rules, you'd be stuck.

There are three distinct mechanisms for this.

### 1. Implicit Conversion — automatic, safe, no syntax needed

The compiler converts automatically when there is zero risk of data loss.

```
int a = 100;  
long b = a;           // int fits inside long -- automatic  
double d = a;         // int fits inside double -- automatic  
float f = a;          // automatic  
  
// Works because the destination type can always hold  
// every possible value of the source type
```

The rule: implicit conversion only happens when the destination type is "wider" than the source. Think of it as pouring water from a small cup into a large cup — it always fits.

```
// This does NOT compile -- going the other way is risky
long big = 9999999999L;
int small = big;    // compile error -- data loss possible
```

## 2. Explicit Conversion (Casting) — manual, you accept the risk

When you're going from a wider type to a narrower one, you must tell the compiler explicitly — you're acknowledging that data loss might happen.

```
double price = 9.99;
int truncated = (int)price;    // explicit cast
Console.WriteLine(truncated); // 9 -- decimal part is silently dropped
```

The syntax is (targetType)value. The compiler trusts you — no error, even if data is lost.

```
long big = 9999999999L;
int small = (int)big;          // compiles, but value will be wrong
Console.WriteLine(small);      // -1294967297 -- overflow, garbage value
```

Casting doesn't round — it truncates. (int)9.99 gives 9, not 10.

### Casting between class types

Casting also applies to class hierarchies — covered properly in section 2.6 (Inheritance). Brief preview:

```
object obj = "hello";          // string stored as object
string s = (string)obj;        // explicit cast back -- safe here

object obj2 = 42;
string s2 = (string)obj2;      // InvalidCastException at runtime
```

Use as for safe casting — returns null instead of throwing:

```
string s = obj as string;      // null if cast fails, no exception
if (s != null) { ... }
```

## 3. Parse and TryParse — converting strings to other types

Implicit and explicit casting don't work on string to numeric conversions. User input always arrives as a string — you need Parse or TryParse.

**Parse** — converts or throws an exception:

```
string input = "42";
int number = int.Parse(input);    // works
double price = double.Parse("9.99"); // works

int bad = int.Parse("hello");      // FormatException -- crashes
int bad2 = int.Parse("9999999999999999"); // OverflowException -- crashes
```

Parse is fine when you control the input. Never use it on user-provided data without validation.

**TryParse** — the safe version for untrusted input:

```
string userInput = "abc";

bool success = int.TryParse(userInput, out int result);
```

```

if (success)
{
    Console.WriteLine($"Parsed: {result}");
}
else
{
    Console.WriteLine("Invalid input -- not a number");
}

```

TryParse returns bool — true if it worked, false if it didn't. The parsed value goes into the out parameter. It never throws — it just returns false on failure.

Shorthand pattern you'll see constantly:

```

if (int.TryParse(userInput, out int value))
{
    // value is available and valid here
}

```

### Convert class — an alternative

System.Convert is a third option, mostly used when converting from object or when you want nulls handled:

```

Convert.ToInt32("42");           // same as int.Parse
Convert.ToInt32(null);           // returns 0 (doesn't throw)
Convert.ToDouble("3.14");

```

In practice, prefer Parse/TryParse for string conversions. Convert appears mainly in older codebases and specific interop scenarios.

### ToString() — going the other way

Every type has ToString() — converts any value to its string representation:

```

int age = 30;
string s = age.ToString();           // "30"

double price = 9.99;
string formatted = price.ToString("C"); // "$9.99" (currency format)
string twoDecimal = price.ToString("F2"); // "9.99"

DateTime now = DateTime.Now;
string date = now.ToString("yyyy-MM-dd"); // "2026-06-29"

```

Format strings are a useful tool — "C" for currency, "F2" for 2 decimal places, "yyyy-MM-dd" for ISO dates.

### Realistic scenario — API input validation

This is exactly how type conversion appears in a real .NET controller:

```

// Query parameter arrives as string: GET /api/orders?page=2&limit=abc
[HttpGet]
public IActionResult GetOrders([FromQuery] string page, [FromQuery] string limit)
{
    if (!int.TryParse(page, out int pageNumber) || pageNumber < 1)
        return BadRequest("Invalid page number");
}

```

```

    if (!int.TryParse(limit, out int pageSize) || pageSize < 1)
        return BadRequest("Invalid limit");

    var orders = _service.GetOrders(pageNumber, pageSize);
    return Ok(orders);
}

```

## Conversion summary

| Scenario                         | Mechanism      | Throws?                     |
|----------------------------------|----------------|-----------------------------|
| int -> long (safe widening)      | Implicit       | No                          |
| double -> int (narrowing)        | Explicit (int) | No -- silently loses data   |
| string -> int (controlled input) | int.Parse()    | Yes on failure              |
| string -> int (user input)       | int.TryParse() | Never                       |
| object -> specific type (safe)   | as operator    | No -- returns null          |
| object -> specific type (unsafe) | Explicit cast  | Yes -- InvalidCastException |

## Common Misconceptions

- **Casting rounds the value.** It doesn't — (int)9.99 is 9, not 10. It truncates toward zero. Use Math.Round() if you want rounding before casting.
- **Parse and TryParse are interchangeable choices.** They're not — Parse is only appropriate when you're certain the input is valid (e.g. reading from your own config file). TryParse is mandatory for any user-provided or external input.
- **Implicit conversion means no conversion is happening.** The compiler is still emitting conversion instructions — it's just doing it for you. Performance is the same as explicit casting.
- **I can cast any type to any other type.** No — casting works between related numeric types and class hierarchies. Casting string to int with (int)myString doesn't compile — that's what Parse is for.
- **Convert.ToInt32 and int.Parse are always equivalent.** Almost, but Convert.ToInt32(null) returns 0 while int.Parse(null) throws. This matters when handling nullable data.

## Practice Problem

A background job reads lines from a CSV file. Each line has a product name and price:

```

Keyboard,1299.99
Monitor,abc
Mouse,799

```

Write a method that takes a string price, safely parses it to decimal, and returns null if it's invalid. Then show how you'd call it for each line and handle the result. Try it before moving on — TryParse exists on decimal too.

## Self-check — you should now be able to explain:

- When implicit conversion happens and what condition must be true
- What explicit casting does to a double like 9.7 when cast to int
- Why TryParse is always preferred over Parse for user input



- What the out keyword does in a TryParse call
- The difference between (string)obj and obj as string

## 1.6. Operators (Arithmetic, Logical, Comparison, Null-Coalescing ?? and ??=)

---

**Builds on:** 1.4 (variables and types), 1.5 (type conversion) — operators act on the values stored in variables.

### What are operators?

Operators are symbols that perform operations on values (operands) and produce a result. You use them constantly — in conditions, calculations, null checks, and assignments. This topic covers the four groups that appear most in real .NET code.

### 1. Arithmetic Operators

Standard math operations on numeric types:

```
int a = 10, b = 3;

Console.WriteLine(a + b); // 13 -- addition
Console.WriteLine(a - b); // 7  -- subtraction
Console.WriteLine(a * b); // 30 -- multiplication
Console.WriteLine(a / b); // 3  -- integer division (truncates!)
Console.WriteLine(a % b); // 1  -- modulus (remainder)
```

Integer division truncates — this surprises people:

```
int result = 7 / 2;           // 3, not 3.5
double result2 = 7.0 / 2;    // 3.5 -- one operand must be double
double result3 = (double)7 / 2; // 3.5 -- explicit cast first
```

Compound assignment — shorthand you'll see everywhere:

```
int x = 10;
x += 5; // x = x + 5 -> 15
x -= 3; // x = x - 3 -> 12
x *= 2; // x = x * 2 -> 24
x /= 4; // x = x / 4 -> 6
x %= 4; // x = x % 4 -> 2
```

Increment and decrement:

```
int i = 5;
i++; // post-increment: use i (5), then increment -> i is now 6
++i; // pre-increment: increment first -> i is now 7, used as 7
i--; // post-decrement
--i; // pre-decrement
```

Pre vs post only matters when the expression's value is used inline:

```
int i = 5;
Console.WriteLine(i++); // prints 5, then i becomes 6
Console.WriteLine(++i); // i becomes 7, then prints 7
```

## 2. Comparison Operators

Produce a bool result. Used in if statements, LINQ Where clauses, and anywhere you make a decision.

```
int a = 10, b = 20;

Console.WriteLine(a == b);    // false -- equal
Console.WriteLine(a != b);    // true  -- not equal
Console.WriteLine(a > b);     // false -- greater than
Console.WriteLine(a < b);     // true  -- less than
Console.WriteLine(a >= b);    // false -- greater than or equal
Console.WriteLine(a <= b);    // true  -- less than or equal
```

String comparison — use == not reference:

```
string s1 = "hello";
string s2 = "hello";

Console.WriteLine(s1 == s2);          // true  -- content compared
Console.WriteLine(s1.Equals(s2));     // true

// Case-insensitive comparison:
Console.WriteLine(string.Equals(s1, "HELLO",
    StringComparison.OrdinalIgnoreCase)); // true
```

C# overloads == for string to compare content, not reference — unlike Java. Still, for case-insensitive or culture-aware comparisons, use string.Equals with a StringComparison argument.

## 3. Logical Operators

Combine bool expressions. Used in if conditions and LINQ filters.

```
bool a = true, b = false;

Console.WriteLine(a && b);    // false -- AND: both must be true
Console.WriteLine(a || b);   // true  -- OR:  at least one true
Console.WriteLine(!a);       // false -- NOT: inverts the value
```

Short-circuit evaluation — critical to understand:

&& stops evaluating as soon as it finds a false. || stops evaluating as soon as it finds a true.

```
// Safe null check -- relies on short-circuit:
if (user != null && user.IsActive)
{
    // user.IsActive is only evaluated if user != null
    // without short-circuit this would NullReferenceException
}

// Same principle with ||:
if (cache == null || cache.IsExpired)
{
    // IsExpired only called if cache is not null
}
```

This pattern — null check before property access — is everywhere in .NET code. Short-circuit is what makes it safe.

#### 4. Null-Coalescing Operators — ?? and ??=

These are C#-specific operators designed specifically for null handling. You will use them constantly.

##### ?? — null coalescing operator

Returns the left side if it's not null, otherwise returns the right side (the fallback).

```
string name = null;
string display = name ?? "Anonymous";
Console.WriteLine(display); // "Anonymous"
```

```
string name2 = "Sathish";
string display2 = name2 ?? "Anonymous";
Console.WriteLine(display2); // "Sathish"
```

Read ?? as: "use this, or if null, use that."

Real scenario — API returning a default when config is missing:

```
// appsettings might not have this key -- returns null
string timeout = configuration["RequestTimeout"];

// Use configured value or fall back to 30 seconds
int seconds = int.Parse(timeout ?? "30");
```

Chaining ?? — falls through until a non-null value is found:

```
string result = firstName ?? middleName ?? lastName ?? "Unknown";
```

##### ??= — null coalescing assignment operator

Assigns the right side to the variable only if the variable is currently null. If it already has a value, nothing happens.

```
string name = null;
name ??= "Default";
Console.WriteLine(name); // "Default"

name ??= "Other"; // name is already "Default", not null
Console.WriteLine(name); // "Default" -- unchanged
```

Read ??= as: "assign this value, but only if I'm currently null."

Real scenario — lazy initialisation:

```
private List<string> _cache;

public List<string> GetCache()
{
    _cache ??= new List<string>(); // initialise only on first call
    return _cache;
}
```

##### Null-conditional operator ?. — closely related

Not null-coalescing but always taught alongside it. Lets you access a member only if the object isn't null — returns null instead of throwing.

```

User user = null;

// Without ?. -- throws NullReferenceException:
string email = user.Email;

// With ?. -- returns null safely:
string email = user?.Email;

// Chain them:
string city = user?.Address?.City;

// Combine with ?? for a final fallback:
string city = user?.Address?.City ?? "Unknown";

```

This trio — `?.`, `??`, `??=` — covers the majority of null-handling patterns in modern C#.

## Operator precedence — practical rules

Full precedence tables exist but you rarely need to memorise them. Two rules cover 95% of cases:

- Arithmetic before comparison: `a + b > c` means `(a + b) > c`
- `&&` before `||`: `a || b && c` means `a || (b && c)`

When in doubt, use parentheses. They cost nothing and make intent explicit:

```

// Unclear:
if (a > 0 || b > 0 && c > 0)

// Clear:
if (a > 0 || (b > 0 && c > 0))

```

## Realistic scenario — combining all four groups

```

public string GetUserDisplayInfo(User user)
{
    // ?. for safe access, ?? for fallback
    string name = user?.FullName ?? "Anonymous";

    // arithmetic + comparison
    int age = user?.Age ?? 0;
    string ageGroup = age >= 18 ? "Adult" : "Minor";

    // logical operator in condition
    bool showEmail = user != null && user.IsVerified;
    string email = showEmail ? user.Email : "Hidden";

    return $"{name} | {ageGroup} | {email}";
}

```

## Common Misconceptions

- **= and == are interchangeable.** `=` assigns a value. `==` compares values. `if (x = 5)` is a bug — it assigns 5 to `x`, not checks if `x` is 5.

- **&& and & are the same.** && short-circuits. & always evaluates both sides (bitwise AND). Use && for boolean logic — always.
- **Integer division gives a decimal result.** It doesn't. 7 / 2 is 3. If you want 3.5, at least one operand must be a floating-point type.
- **?? works on value types.** It doesn't — value types can't be null (unless nullable: int?). ?? only applies to reference types and nullable value types.
- **?. and ?? do the same thing.** ?. navigates safely (returns null on null object). ?? provides a fallback when something IS null. They're complementary, not alternatives.

## Practice Problem

What does this method return for each call, and why?

```
public string Describe(int? score, string label)
{
    string displayLabel = label ?? "Unranked";
    int actualScore = score ?? 0;
    bool isPassing = actualScore >= 50 && actualScore <= 100;
    string status = isPassing ? "Pass" : "Fail";
    return $"{displayLabel}: {actualScore} ({status})";
}

Describe(75, "Math");
Describe(null, "Science");
Describe(30, null);
Describe(null, null);
```

Write the four outputs before moving on.

## Self-check — you should now be able to explain:

- Why 7 / 2 gives 3 and how to get 3.5
- What short-circuit evaluation is and why user != null && user.IsActive is safe
- The difference between ?? and ??= with an example of each
- What ?. returns when the object is null vs when it isn't
- Why && should always be used instead of & for boolean conditions

## 1.7. Control Flow — if/else, Switch Statements and Expressions

**Builds on:** 1.6 — comparison and logical operators produce the bool conditions that control flow depends on entirely.

### What is control flow and why does it exist?

Without control flow, every program runs top to bottom — the same path every time regardless of data. Control flow lets your program make decisions and take different paths based on conditions. It's the difference between a program that always does the same thing and one that responds to real-world data.

## if / else if / else

The fundamental decision structure. Evaluates a bool condition and runs the matching block.

```
int score = 72;

if (score >= 90)
{
    Console.WriteLine("A");
}
else if (score >= 80)
{
    Console.WriteLine("B");
}
else if (score >= 70)
{
    Console.WriteLine("C");
}
else
{
    Console.WriteLine("Fail");
}
// prints: C
```

Rules:

- Only the first matching branch executes — the rest are skipped
- else if and else are optional
- Conditions are evaluated top to bottom — order matters

## Ternary operator — inline if/else

For simple two-way decisions that produce a value:

```
int age = 20;
string label = age >= 18 ? "Adult" : "Minor";
```

Read as: if age >= 18, use "Adult", otherwise use "Minor".

Real use in .NET — returning different status codes or labels inline:

```
return user != null ? Ok(user) : NotFound();
```

Don't nest ternaries — they become unreadable fast. If you need more than one level, use if/else.

## switch statement — matching one value against many cases

When you're checking one variable against multiple specific values, switch is cleaner than a chain of if/else if.

```
string role = "Admin";

switch (role)
{
    case "Admin":
        Console.WriteLine("Full access");
        break;
    case "Editor":
```

```

        Console.WriteLine("Edit access");
        break;
    case "Viewer":
        Console.WriteLine("Read only");
        break;
    default:
        Console.WriteLine("No access");
        break;
}

```

Important rules:

- break is mandatory at the end of each case — C# does not fall through automatically
- default runs if no case matches — equivalent to else
- Multiple cases can share the same block:

```

case "Admin":
case "SuperAdmin":
    Console.WriteLine("Full access");
    break;

```

### switch expression — modern C# (prefer this)

Introduced in C# 8. More concise, produces a value directly, no break needed. This is what you'll see in modern .NET codebases.

```

string role = "Admin";

string access = role switch
{
    "Admin"      => "Full access",
    "Editor"     => "Edit access",
    "Viewer"     => "Read only",
    _            => "No access"    // _ is the discard -- matches anything
};

Console.WriteLine(access);

```

The `_` wildcard replaces default. Each arm is pattern => result. No break, no case keyword.

### Pattern matching in switch expressions

Switch expressions become significantly more powerful with patterns — you can match on type, property values, and conditions, not just exact values.

Type patterns:

```

object shape = new Circle { Radius = 5 };

string description = shape switch
{
    Circle c      => $"Circle with radius {c.Radius}",
    Rectangle r   => $"Rectangle {r.Width}x{r.Height}",
    _              => "Unknown shape"
};

```

Property patterns — match based on property values:

```

var order = new Order { Status = "Pending", Amount = 150 };

string message = order switch
{
    { Status: "Pending", Amount: > 100 } => "Large pending order",
    { Status: "Pending" }                  => "Small pending order",
    { Status: "Shipped" }                  => "Order on its way",
    _                                       => "Other status"
};

```

Relational patterns — use comparisons directly:

```

int score = 85;

string grade = score switch
{
    >= 90 => "A",
    >= 80 => "B",
    >= 70 => "C",
    >= 60 => "D",
    _     => "F"
};

```

This replaces the chain of if/else if from earlier — much cleaner when all you're doing is mapping a value to a result.

## When to use which

| Situation                                  | Use                                  |
|--------------------------------------------|--------------------------------------|
| Simple two-way decision                    | if/else or ternary                   |
| Matching one value to many exact options   | switch expression                    |
| Complex conditions with multiple variables | if/else if chain                     |
| Mapping a value to a result                | switch expression                    |
| Type-based branching                       | switch expression with type patterns |

Prefer switch expressions over switch statements in new code — they're more concise, enforce exhaustiveness, and produce a value directly.

## Realistic scenario — controller action logic

```

[HttpGet("{id}")]
public IActionResult GetOrder(int id)
{
    var order = _repo.FindById(id);

    if (order == null)
        return NotFound();

    string statusMessage = order.Status switch
    {
        "Pending"    => "Your order is being processed",
        "Shipped"    => "Your order is on its way",
        "Delivered"  => "Your order has been delivered",
        "Cancelled"  => "Your order was cancelled",
        _            => "Unknown status"
    };
}

```



```

};

return Ok(new { order, statusMessage });
}

```

## Guard clauses — a pattern worth knowing now

Rather than deeply nesting if/else, return early when conditions fail. This keeps the happy path at the left margin and is considered good practice in .NET codebases.

```

// Nested -- hard to read as logic grows:
public IActionResult Process(Order order)
{
    if (order != null)
    {
        if (order.IsValid)
        {
            if (order.Amount > 0)
            {
                // actual logic buried here
            }
        }
    }
    return BadRequest();
}

// Guard clauses -- flat and readable:
public IActionResult Process(Order order)
{
    if (order == null)      return BadRequest("Order is null");
    if (!order.IsValid)    return BadRequest("Order is invalid");
    if (order.Amount <= 0) return BadRequest("Amount must be positive");

    // happy path -- no nesting
    _service.Process(order);
    return Ok();
}

```

You'll see guard clauses in virtually every well-written ASP.NET Core controller.

## Common Misconceptions

- **switch falls through like in C or JavaScript.** It doesn't in C#. Each case requires an explicit break (or return, throw). The compiler enforces this — forgetting break is a compile error, not a silent bug.
- **The ternary operator can replace any if/else.** Only for simple two-way value assignments. Never use nested ternaries — they're nearly unreadable.
- **switch expression \_ and default are the same.** Conceptually yes — both are catch-alls. But in a switch expression, if you omit \_ and a value doesn't match any arm, you get an `InvalidOperationException` at runtime. Always include \_.
- **if/else and switch are equally readable for many conditions.** For 5+ conditions on one variable, a switch expression is dramatically cleaner. An if/else if chain of 8 conditions is a code smell.

- **Guard clauses are just a style preference.** They meaningfully reduce cognitive load. Most .NET style guides and code reviews expect them.

### Practice Problem

Write a method `GetShippingCost(string country, decimal orderAmount)` that returns a decimal shipping cost using these rules:

- India: free if order  $\geq 500$ , otherwise 49
- USA: free if order  $\geq 2000$ , otherwise 299
- Any other country: always 499

Use a switch expression where appropriate. Try it before moving on.

### Self-check — you should now be able to explain:

- The difference between switch statement and switch expression and when to prefer each
- What the `_` discard pattern means in a switch expression
- What a guard clause is and why it's preferred over deep nesting
- How property patterns work in a switch expression with a concrete example
- Why `break` is required in a switch statement but not in a switch expression

## 1.8. Loops — for, foreach, while, do-while

---

**Builds on:** 1.7 (control flow — loops are control flow that repeats), 1.4 (variables — loop counters and iteration variables are variables).

### What are loops and why do they exist?

Without loops, repeating an operation 1000 times means writing the same line 1000 times. Loops let you express "do this repeatedly" — either a fixed number of times, over a collection of items, or until a condition changes. They're one of the most used constructs in any program.

C# has four loop types. Each exists because it fits a specific shape of problem better than the others.

#### 1. for loop — when you know the count upfront

Use when you know exactly how many times to iterate, or when you need the index.

```
for (int i = 0; i < 5; i++)
{
    Console.WriteLine(i);
}
// prints: 0 1 2 3 4
```

Three parts in the header:

- `int i = 0` — initialiser, runs once before the loop starts
- `i < 5` — condition, checked before each iteration
- `i++` — iterator, runs after each iteration

All three parts are optional — `for(;;)` is an infinite loop.

Iterating backwards:

```
for (int i = 4; i >= 0; i--)
{
    Console.WriteLine(i);
}
// prints: 4 3 2 1 0
```

Using the index to access an array:

```
string[] names = { "Sathish", "Priya", "Karthik" };

for (int i = 0; i < names.Length; i++)
{
    Console.WriteLine($"{i}: {names[i]}");
}
// 0: Sathish
// 1: Priya
// 2: Karthik
```

Use `for` when you need the index. If you don't need the index, `foreach` is cleaner.

## 2. `foreach` — when iterating a collection

The most common loop in .NET codebases. Iterates over any collection that implements `IEnumerable` — arrays, `List`, results of LINQ queries, EF Core query results, and more.

```
var names = new List<string> { "Sathish", "Priya", "Karthik" };

foreach (string name in names)
{
    Console.WriteLine(name);
}
```

Cleaner than `for` when you don't need the index — no counter to manage, no off-by-one errors, no `.Length` to check.

`foreach` with LINQ result — the pattern you'll use constantly:

```
var activeUsers = users.Where(u => u.IsActive); // returns IEnumerable<User>

foreach (var user in activeUsers)
{
    Console.WriteLine(user.Name);
}
```

`var in foreach` — always fine:

```
foreach (var order in orders) // compiler infers Order type
```

Critical rule — do not modify the collection inside `foreach`:

```
var list = new List<int> { 1, 2, 3, 4 };

foreach (var item in list)
{
    if (item == 2)
        list.Remove(item); // InvalidOperationException at runtime
}
```

The enumerator that `foreach` uses internally detects the modification and throws. If you need to remove items while iterating, use a `for` loop iterating backwards, or collect items to remove and delete them after the loop.

### 3. while — when the count is unknown, condition checked first

Use when you don't know upfront how many iterations you need — you loop until something changes.

```
int number = 1;

while (number < 100)
{
    number *= 2;
}

Console.WriteLine(number); // 128
```

The condition is checked before each iteration. If it's false on entry, the body never runs.

Real scenario — polling or retry logic:

```
int attempts = 0;
bool success = false;

while (!success && attempts < 3)
{
    success = TrySendEmail();
    attempts++;
}
```

### 4. do-while — condition checked after, body always runs at least once

Identical to `while` except the condition is evaluated after the body. Guarantees at least one execution.

```
int input;

do
{
    Console.Write("Enter a positive number: ");
    input = int.Parse(Console.ReadLine());
}
while (input <= 0);
```

The body runs once, then the condition decides if it continues. In practice, `do-while` is the least used loop — the guarantee of one execution is only occasionally what you need.

### Loop control — break and continue

`break` — exits the loop immediately:

```
foreach (var user in users)
{
    if (user.Id == targetId)
    {
        Console.WriteLine($"Found: {user.Name}");
        break;
    }
}
```

```

        break; // stop searching once found
    }
}

```

continue — skips the rest of the current iteration, moves to the next:

```

foreach (var order in orders)
{
    if (order.IsCancelled)
        continue; // skip cancelled orders

    ProcessOrder(order); // only runs for non-cancelled
}

```

Both are legitimate tools. break is common in search loops. continue is common as a guard at the top of a loop body.

## Choosing the right loop

| Situation                       | Use      |
|---------------------------------|----------|
| Known count, or need index      | for      |
| Iterating any collection        | foreach  |
| Unknown count, condition-driven | while    |
| Must run at least once          | do-while |

In practice, foreach covers ~70% of loop use cases in .NET backend code. for covers most of the rest. while appears in retry logic, polling, and reading streams. do-while is rare.

## Realistic scenario — processing API results

```

[HttpPost("process-orders")]
public IActionResult ProcessOrders()
{
    var orders = _repo.GetPendingOrders();
    var processedIds = new List<int>();
    var failedIds = new List<int>();

    foreach (var order in orders)
    {
        if (order.Amount <= 0)
            continue; // skip invalid orders

        bool success = _paymentService.Charge(order);

        if (success)
            processedIds.Add(order.Id);
        else
            failedIds.Add(order.Id);
    }

    return Ok(new { processedIds, failedIds });
}

```

## A note on LINQ vs loops

From section 5 onwards, you'll learn LINQ — which lets you replace many loops with cleaner declarative expressions:

```
// Loop version:
var result = new List<string>();
foreach (var user in users)
{
    if (user.IsActive)
        result.Add(user.Name);
}

// LINQ version (same result):
var result = users.Where(u => u.IsActive).Select(u => u.Name).ToList();
```

LINQ doesn't eliminate loops — it's built on `foreach` internally. But for transforming and filtering collections, LINQ is preferred in modern .NET code. You still need raw loops for side-effectful operations (like charging payments, sending emails, writing files).

### Common Misconceptions

- **foreach is slower than for.** For most collection types in .NET, the performance difference is negligible. Prefer readability. Only optimise with `for` if profiling proves a bottleneck.
- **I can modify a collection inside foreach.** You cannot — it throws `InvalidOperationException`. Collect items to modify, then act on them after the loop.
- **while(true) is always bad.** It's fine when paired with a `break` — common in consumer loops reading from a queue or stream. The infinite loop is intentional, `break` is the exit.
- **do-while and while are interchangeable.** They differ on first execution. For most cases `while` is correct. Use `do-while` only when the body must run at least once regardless of the condition.
- **You need a loop to process every collection.** Not in modern C#. LINQ handles filtering, projection, and aggregation without explicit loops.

### Practice Problem

You have a list of integers:

```
var numbers = new List<int> { 3, 7, -2, 15, 0, 8, -5, 42, 1 };
```

Write a loop (using `foreach` and `continue`) that:

- Skips negative numbers and zero
- Prints each valid number
- Stops entirely if it encounters a number greater than 20

Expected: prints 3, 7, 15 — then stops at 42 without printing it.

### Self-check — you should now be able to explain:

- When to choose `for` over `foreach` and why
- Why modifying a collection inside `foreach` throws an exception
- The one meaningful difference between `while` and `do-while`
- What `break` and `continue` do and give a real use case for each

- Why foreach covers most loop cases in .NET backend code

## 1.9. Arrays (Single, Multi-Dimensional, Jagged)

---

**Builds on:** 1.4 (variables and types), 1.8 (loops — you almost always iterate arrays with loops).

### What are arrays and why do they exist?

An array is a fixed-size, ordered collection of elements of the same type stored in contiguous memory. The key word is fixed — once created, the size cannot change.

Without arrays, storing 100 user IDs would mean declaring 100 separate variables. Arrays let you store multiple values of the same type under one name and access them by position.

In real .NET code, List (section 4.1) is used far more than arrays because it's resizable. But arrays underlie everything — List uses an array internally, method parameters often use arrays, and understanding arrays is prerequisite to understanding collections.

### Single-Dimensional Arrays

Declaration and initialisation:

```
// Declare with size -- elements get default values (0 for int, null for string)
int[] scores = new int[5];           // [0, 0, 0, 0, 0]

// Declare and initialise with values
int[] scores = new int[] { 10, 20, 30, 40, 50 };

// Shorthand -- compiler infers size
int[] scores = { 10, 20, 30, 40, 50 };

// var works too
var scores = new int[] { 10, 20, 30, 40, 50 };
```

Accessing elements — zero-indexed:

```
int[] scores = { 10, 20, 30, 40, 50 };

Console.WriteLine(scores[0]); // 10 -- first element
Console.WriteLine(scores[4]); // 50 -- last element
Console.WriteLine(scores[5]); // IndexOutOfRangeException

// Length property:
Console.WriteLine(scores.Length); // 5
```

Arrays are zero-indexed — first element is index 0, last is Length - 1. Accessing beyond that throws IndexOutOfRangeException.

Modifying elements:

```
scores[0] = 99; // replace first element
```

Iterating — both styles:

```
// foreach -- when you don't need the index
foreach (int score in scores)
{
```

```

        Console.WriteLine(score);
    }

    // for -- when you need the index
    for (int i = 0; i < scores.Length; i++)
    {
        Console.WriteLine($"Index {i}: {scores[i]}");
    }

```

### Index from end — ^ operator (C# 8+)

A cleaner way to access elements from the end without calculating Length - 1:

```

int[] scores = { 10, 20, 30, 40, 50 };

Console.WriteLine(scores[^1]); // 50 -- last element
Console.WriteLine(scores[^2]); // 40 -- second to last

```

^1 means "1 from the end." This is syntactic sugar — the compiler converts it to scores[scores.Length - 1].

### Range operator — .. (C# 8+)

Slice an array to get a sub-array:

```

int[] scores = { 10, 20, 30, 40, 50 };

int[] first3 = scores[0..3]; // { 10, 20, 30 } -- indices 0, 1, 2 (3 is excluded)
int[] last2 = scores[^2..]; // { 40, 50 }
int[] middle = scores[1..4]; // { 20, 30, 40 }

```

The range start..end is inclusive of start, exclusive of end. These return new arrays — the original is unchanged.

### Useful array methods

```

int[] nums = { 5, 2, 8, 1, 9, 3 };

Array.Sort(nums);
// nums is now { 1, 2, 3, 5, 8, 9 } -- sorts IN PLACE

Array.Reverse(nums);
// nums is now { 9, 8, 5, 3, 2, 1 } -- reverses IN PLACE

int idx = Array.IndexOf(nums, 5);
// returns index of value 5, or -1 if not found

int[] copy = new int[nums.Length];
Array.Copy(nums, copy, nums.Length);
// copies contents -- copy is independent

```

Important: Sort and Reverse modify the array directly — they don't return a new one.

### Multi-Dimensional Arrays



A true rectangular grid — every row has the same number of columns. Think of a spreadsheet or a game board.

```
// 3 rows, 4 columns
int[,] grid = new int[3, 4];

// Declare and initialise
int[,] matrix = {
    { 1, 2, 3 },
    { 4, 5, 6 },
    { 7, 8, 9 }
};

// Access with two indices [row, column]
Console.WriteLine(matrix[0, 0]); // 1 -- top-left
Console.WriteLine(matrix[1, 2]); // 6 -- row 1, column 2
Console.WriteLine(matrix[2, 2]); // 9 -- bottom-right

// Dimensions:
Console.WriteLine(matrix.GetLength(0)); // 3 -- row count
Console.WriteLine(matrix.GetLength(1)); // 3 -- column count

// Iterate with nested for loops:
for (int row = 0; row < matrix.GetLength(0); row++)
{
    for (int col = 0; col < matrix.GetLength(1); col++)
    {
        Console.Write($"{matrix[row, col]} ");
    }
    Console.WriteLine();
}
```

Where you'll see this in practice: matrix operations, game grids, seating charts, image pixel buffers. Less common in typical API/backend work — List<T> or a flat list with computed indices is more common.

### Jagged Arrays — arrays of arrays

Each "row" is a separate array that can have a different length. Hence "jagged" — the rows don't have to line up.

```
// Declare a jagged array with 3 rows
int[][] jagged = new int[3][];

// Each row is its own array with its own size
jagged[0] = new int[] { 1, 2 };
jagged[1] = new int[] { 3, 4, 5, 6 };
jagged[2] = new int[] { 7 };

// Access with two separate [] operators
Console.WriteLine(jagged[0][1]); // 2
Console.WriteLine(jagged[1][3]); // 6
Console.WriteLine(jagged[2][0]); // 7

// Iterate:
```

```

for (int i = 0; i < jagged.Length; i++)
{
    for (int j = 0; j < jagged[i].Length; j++)
    {
        Console.Write($"{jagged[i][j]} ");
    }
    Console.WriteLine();
}

```

Multi-dimensional vs jagged:

|            | Multi-dimensional [,] | Jagged [][]                   |
|------------|-----------------------|-------------------------------|
| Shape      | Rectangular           | Irregular -- rows can differ  |
| Syntax     | matrix[row, col]      | jagged[row][col]              |
| Memory     | One contiguous block  | Each row separate allocation  |
| Common use | Grids, matrices       | Grouped data of varying sizes |

In practice, jagged arrays appear when you have groups of different sizes — e.g. questions per quiz section, students per class.

### Arrays as reference types

Arrays are reference types — assigning one array variable to another copies the reference, not the data:

```

int[] a = { 1, 2, 3 };
int[] b = a;           // b points to the SAME array

b[0] = 99;
Console.WriteLine(a[0]); // 99 -- a was affected

```

To get an independent copy:

```

int[] b = (int[])a.Clone(); // shallow copy
// or
int[] b = new int[a.Length];
Array.Copy(a, b, a.Length);

```

### Arrays in real .NET API code

```

// Method that accepts variable number of IDs
[HttpPost("bulk-delete")]
public IActionResult BulkDelete([FromBody] int[] ids)
{
    if (ids == null || ids.Length == 0)
        return BadRequest("No IDs provided");

    foreach (var id in ids)
    {
        _repo.Delete(id);
    }

    return NoContent();
}

```

Arrays are common in: method parameters accepting a fixed set of values, string[] args in the entry point (1.3), JSON deserialization of arrays from request bodies, returning fixed sets of

allowed values.

### Common Misconceptions

- **Arrays and List are interchangeable.** Arrays are fixed-size — you cannot add or remove elements. List is resizable. For most collection work in APIs, use List. Use arrays when size is known and fixed.
- **Array index starts at 1.** It starts at 0. First element is [0], last is [Length - 1]. Off-by-one errors from assuming 1-based indexing are a classic beginner mistake.
- **Multi-dimensional and jagged arrays are the same.** They're different in syntax, memory layout, and use case. `int[,]` is rectangular and contiguous. `int[][]` is an array of independently allocated arrays.
- **Assigning one array to another copies the data.** It copies the reference — both variables point to the same data. Mutations through either variable affect the same underlying array.
- **Array.Sort() returns a sorted copy.** It sorts the original array in place and returns void. If you need the original preserved, copy it first.

### Practice Problem

You have this array of temperatures recorded over a week:

```
double[] temps = { 36.5, 37.1, 36.8, 38.2, 37.5, 36.9, 37.8 };
```

Write code that:

- Prints the highest and lowest temperature
- Prints the average temperature rounded to 1 decimal place
- Prints how many days had a temperature above 37.0

Do it with a loop — no LINQ yet. Try before moving on.

### Self-check — you should now be able to explain:

- Why arrays are zero-indexed and what happens if you access an out-of-range index
- The difference between multi-dimensional `[,]` and jagged `[][]` arrays with a use case for each
- Why assigning an array to another variable doesn't copy the data and how to actually copy it
- When to use an array vs a List (conceptually — you'll cover List properly in section 4.1)
- What `^1` and `1..4` do as index and range operators

## 1.10. Strings — Manipulation, Interpolation, StringBuilder

---

**Builds on:** 1.4 (string is a type), 1.8 (loops — iterating characters), 1.9 (strings are internally arrays of chars).

### What is a string in C#?

A string is a sequence of Unicode characters. Under the hood it's essentially a `char[]` — but with a rich API built on top and one critical property: immutability.

Once a string is created, it cannot be changed. Every operation that appears to modify a string actually creates a new one.

```
string name = "Sathish";
name = name + " Kumar";    // "Sathish" is not modified
                           // a NEW string "Sathish Kumar" is created
                           // name now points to the new string
```

This matters for performance when you're doing many string operations in a loop — covered at the end with `StringBuilder`.

## Creating and accessing strings

```
string s = "Hello, World!";

// Length
Console.WriteLine(s.Length);    // 13

// Access individual characters -- strings are zero-indexed like arrays
Console.WriteLine(s[0]);        // 'H'
Console.WriteLine(s[^1]);       // '!' -- last char (range operator from 1.9)

// Iterate characters
foreach (char c in s)
{
    Console.Write(c + " ");
}
// H e l l o ,   W o r l d !
```

## String interpolation — the way to build strings in modern C#

Prefix the string with `$` and embed expressions directly in `{}`:

```
string name = "Sathish";
int age = 30;
decimal salary = 85000.50m;

string message = $"Name: {name}, Age: {age}, Salary: {salary:C}";
// "Name: Sathish, Age: 30, Salary: $85,000.50"
```

Format specifiers inside `{}`:

- `{value:C}` — currency format
- `{value:F2}` — 2 decimal places
- `{value:D5}` — integer padded to 5 digits (e.g. 00042)
- `{value:yyyy-MM-dd}` — date format

```
DateTime now = DateTime.Now;
int orderId = 42;

string log = $"[{now:yyyy-MM-dd HH:mm:ss}] Order {orderId:D6} processed";
// "[2026-06-29 14:30:00] Order 000042 processed"
```

Interpolation replaced `string.Format()` in modern C#. You'll still see `string.Format("Hello {0}", name)` in older codebases — they're equivalent, interpolation is just cleaner.

## Essential string methods

Checking content:

```
string s = "Hello, World!";

s.Contains("World")           // true
s.StartsWith("Hello")        // true
s.EndsWith("!")              // true
s.IsNullOrEmpty(s)          // false -- has content
string.IsNullOrEmpty(" ")    // true -- only spaces
```

Changing case:

```
"hello".ToUpper()    // "HELLO"
"HELLO".ToLower()    // "hello"
```

Trimming whitespace:

```
" hello ".Trim()      // "hello"
" hello ".TrimStart() // "hello "
" hello ".TrimEnd()   // " hello"
```

Finding and extracting:

```
string s = "Hello, World!";

s.IndexOf("World")      // 7 -- position of first match, -1 if not found
s.Substring(7, 5)       // "World" -- start index, length
s[7..12]                // "World" -- same with range operator (cleaner)
```

Replacing:

```
"Hello, World!".Replace("World", "Sathish") // "Hello, Sathish!"
"aabbcc".Replace("bb", "")                 // "aacc" -- remove by replacing with ""
```

Splitting and joining:

```
// Split -- string to array
string csv = "Sathish,Priya,Karthik";
string[] names = csv.Split(',');
// { "Sathish", "Priya", "Karthik" }

// Split with multiple delimiters
string data = "one;two,three|four";
string[] parts = data.Split(new char[] { ';', ',', '|' });

// Join -- array back to string
string joined = string.Join(", ", names);
// "Sathish, Priya, Karthik"

string joined2 = string.Join(" | ", new[] { "A", "B", "C" });
// "A | B | C"
```

Split and Join are a frequently used pair — parsing CSV-like input, building query strings, formatting lists for display.

## String comparison — do it right

```
string a = "hello";
string b = "HELLO";
```

```
// Wrong for case-insensitive comparison:
a == b // false

// Correct:
string.Equals(a, b, StringComparison.OrdinalIgnoreCase) // true

// Also correct:
a.Equals(b, StringComparison.OrdinalIgnoreCase) // true

// For sorting/ordering (locale-aware):
string.Compare(a, b, StringComparison.CurrentCultureIgnoreCase)
```

Always use `StringComparison` when case-insensitivity matters. In API code this comes up constantly — comparing user-provided role names, matching config keys, searching names.

### Null and empty — always check before calling methods

```
string s = null;
s.Length // NullReferenceException

// Safe checks:
string.IsNullOrEmpty(s) // true -- null or ""
string.IsNullOrWhiteSpace(s) // true -- null, "", or " "

// Null-safe access with ?. from 1.6:
int? len = s?.Length; // null, not an exception
```

In practice, always use `IsNullOrWhiteSpace` over `IsNullOrEmpty` for user input — a string of spaces is effectively empty from a business logic perspective.

### Verbatim strings — @

Prefix with `@` to disable escape sequence processing. Useful for file paths and multi-line strings:

```
// Without @ -- backslashes need escaping
string path = "C:\\Users\\Sathish\\Documents\\file.txt";

// With @ -- backslashes are literal
string path = @"C:\Users\Sathish\Documents\file.txt";

// Multi-line verbatim string
string sql = @"
    SELECT *
    FROM Users
    WHERE IsActive = 1
";
```

Combine `$` and `@` for interpolated verbatim strings:

```
string folder = "Documents";
string path = @$"C:\Users\Sathish\{folder}\file.txt";
```

### Raw string literals — """ (C# 11+)

For strings containing quotes, braces, or complex content without any escaping:

```
string json = """
```

```
{
    "name": "Sathish",
    "role": "Admin"
}
""";
```

No escape characters needed at all. The indentation of the closing `"""` determines how much leading whitespace is stripped. Excellent for embedded JSON, SQL, or HTML in tests and seeding data.

## StringBuilder — when immutability becomes a problem

Because every string operation creates a new string, concatenating in a loop is expensive:

```
// BAD -- creates thousands of intermediate strings:
string result = "";
for (int i = 0; i < 10000; i++)
{
    result += i.ToString();    // new string object on every iteration
}
```

With 10,000 iterations, this allocates roughly 10,000 intermediate strings, most immediately garbage collected. For large loops this is a real performance problem.

StringBuilder solves this — it maintains an internal mutable buffer and only produces a final string when you call `ToString()`:

```
// GOOD -- single mutable buffer:
var sb = new StringBuilder();

for (int i = 0; i < 10000; i++)
{
    sb.Append(i);
}

string result = sb.ToString();    // one string created at the end
```

Common StringBuilder methods:

```
var sb = new StringBuilder();

sb.Append("Hello");                // append text
sb.AppendLine(", World!");          // append with newline
sb.Insert(5, " there");             // insert at position
sb.Replace("World", "Sathish");     // replace within buffer
sb.Remove(0, 5);                   // remove characters

Console.WriteLine(sb.ToString());
```

When to use StringBuilder vs interpolation:

| Scenario                                  | Use                |
|-------------------------------------------|--------------------|
| Building a string from a few known parts  | \$ interpolation   |
| Concatenating in a loop (many iterations) | StringBuilder      |
| Building SQL or large text dynamically    | StringBuilder      |
| Simple one-liners                         | + or interpolation |

The rule of thumb: if you're concatenating strings in a loop with more than a handful of iterations, use `StringBuilder`.

### Realistic scenario — building a CSV export

```
public string ExportOrdersToCsv(List<Order> orders)
{
    var sb = new StringBuilder();

    // Header row
    sb.AppendLine("Id, CustomerName, Amount, Status");

    foreach (var order in orders)
    {
        // Escape commas in names
        string safeName = order.CustomerName.Replace(",", " ");

        sb.AppendLine($"{order.Id}, {safeName}, {order.Amount:F2}, {order.Status}");
    }

    return sb.ToString();
}
```

String interpolation for each line, `StringBuilder` for the loop — the right tool for each job.

### Common Misconceptions

- **Strings are mutable — I can change characters with `s[0] = 'X'`.** You can't — strings are immutable and the indexer is read-only. Use `StringBuilder` or create a new string if you need mutation.
- **`==` always compares references for strings.** No — C# overloads `==` for string to compare content. `"hello" == "hello"` is true. For objects in general, `==` compares references, but strings are a special case.
- **String concatenation with `+` in a loop is fine.** For a handful of items yes, but in a loop with many iterations it causes serious allocation pressure. Use `StringBuilder`.
- **`IsNullOrEmpty` and `IsNullOrWhiteSpace` are interchangeable.** `IsNullOrEmpty` returns false for `" "` (spaces only). `IsNullOrWhiteSpace` returns true. For user input validation, almost always use `IsNullOrWhiteSpace`.
- **Interpolation is slower than concatenation.** The compiler compiles simple interpolations to efficient string operations. Don't avoid it for performance reasons — only switch to `StringBuilder` in tight loops.

### Practice Problem

You receive a raw string of user-submitted tags for a blog post:

```
string input = " C#, .NET , api, C# , web ";
```

Write code that:

- Splits by comma
- Trims each tag



- Converts to lowercase
- Removes duplicates
- Joins the result back with " | " as separator

Expected output: "c# | .net | api | web"

Try it before moving on. Hint: you can use a HashSet to deduplicate (works like a List but ignores duplicate adds — you'll cover it properly in section 4.3).

**Self-check — you should now be able to explain:**

- Why strings are immutable and what that means when you "modify" one
- When to use StringBuilder instead of + or interpolation
- The difference between IsNullOrEmpty and IsNullOrWhiteSpace and which to use for user input
- How to do a case-insensitive string comparison correctly
- What Split and Join do and give a real scenario where you'd use them together